

Cody Researcher : Design Note

1. The problem

- Given a company and a question, find the official site, work out which pages actually matter, read them, and answer with real citations.
- No hard-coding for one specific website, no crawling forever, and never stating something the pages don't actually support.
- The hard part isn't calling a browser tool, it's deciding when to stop, what to trust, and keeping it fast and cheap at the same time.

2. Pipeline shape

- **Resolve:** turns whatever the user typed into a starting URL and an allowed domain.
- **Seed:** reads the homepage and fires one search built from the task itself, usually finds the right deep page in one hop instead of several clicks.
- **Crawl:** fetches a small batch of queued pages at a time, pulls facts from each with an AI call, and queues any new relevant links.
- **Decide:** a plain-code checkpoint — keep going, or stop, based on the queue and the guardrails.
- **Synthesize:** writes the final answer with citation markers from the evidence collected so far.
- **Validate:** plain code again, checks every citation marker points at something real before the answer goes out.
- Batching the crawl (instead of crawling everything, then asking one giant question) lets the agent stop as soon as it has enough, and keeps the guardrail checks running often instead of only once at the very end.

3. Where the AI is used, and where it isn't

- Four model calls do real reasoning: picking the official site, deciding which links are worth following, pulling facts out of a page, and writing the final answer.
- Everything else — cleaning HTML, extracting links, deduplicating URLs, retrying a failed fetch, checking a citation is plain code.
- Roughly one AI call happens per page fetched, plus one per batch of newly found links not one per link, not one per sentence.

4. Tool choices

- **Playwright** : For fallback, used when a plain HTTP fetch fails or looks broken (JS-heavy pages). Not used by default.
- **React + Tailwind (Vite build)** : Frontend for project.
- **LangGraph** : A plain loop can't pause and resume on its own, I'd have to build that myself. LangGraph gives resumability if something gets disrupted/paused.

Role in Cody Researcher : It runs the pipeline as a state graph (resolve → seed → crawl → decide → synthesize → validate → finalize)

- **SQLite** — This runs on one person's machine, not a server with many users. Postgres would mean an extra thing to install and configure which is not needed here. SQLite just works with zero setup.

5. Guardrails

- **Page count**: caps total browsing per task.
- **Iteration count**: catches loop bugs independent of page count.
- **AI-call count**: caps spend directly.
- **Wall-clock time**: catches slow sites that would otherwise hang.
- **Retry count**: bounds how much effort goes into one stubborn URL.
- **Concurrency cap**: keeps resource use and site load under control.

6. Known issues found during development

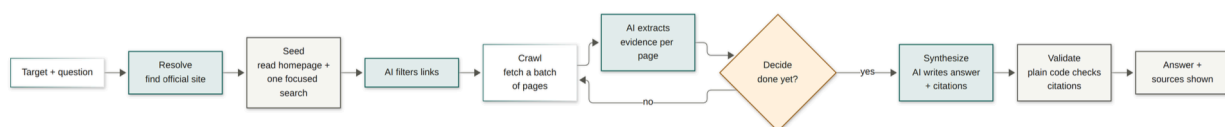
- **413 crash on synthesis**: The first Siemens run collected 34 pages of evidence and the synthesis prompt exceeded Groq's 8,000 limit.
- **86 API calls for a single run**: The `@retry` was retrying non-retryable errors like 413 (payload too large) 3 times, and `ask_json` added another retry layer on top. So, one failed call cost up to 6 actual API calls. Fixed by adding a `retry_if_exception` filter that skips 400/413/422 errors.
- **Playwright launching for 404 pages**: Every HTTP error (including genuine 404s) triggered a full Chromium launch. A 404 page won't magically have content in a browser. Changed to only use Playwright when HTTP succeeds but the body looks empty (JS-rendered apps).

- **Guardrails too generous for free tier:** Default limits (40 pages, 15 iterations, 40 LLM calls) were fine for paid API tiers but caused the agent to accumulate too much evidence on the free tier before hitting synthesis. Lowered to 15 pages, 8 iterations, 20 LLM calls.
- **No rate limiting between API calls:** Burst calls would hit the tokens-per-minute ceiling. Added a 2-second minimum interval between Groq calls.
- **Page text too large for evidence extraction:** Sending 6,000 characters per page to the evidence extraction prompt was expensive.

7. Citation

- Evidence is tagged, stated, extracted, or inferred at the point it's pulled from a page.
- Plain code checks every citation marker in the final answer against evidence actually collected in that run. A marker that doesn't match is flagged, not trusted.
- Every evidence row keeps its source URL, fetch time, and fetch method, so any claim traces back to a specific page.

Pipeline flow diagram



8. What changes at real production scale

- **Database:** Move from SQLite to Postgres for the app's tables, the schema is already plain SQL, so this is just a connection change, not a rewrite.
- **Background jobs:** The single background task becomes a real job queue, so a run survives a restart and multiple runs don't fight over one process.
- **Crawling manners:** Add rate limits per website and a robots.txt check before crawling, not done yet.

- **File types:** Add a way to read PDFs and other non-HTML files, right now they just count as failed pages.
- **Splitting the load:** Move the page-crawling work to its own separate process, so a slow research job can't freeze the part of the app checking run status.

9. Honest gaps in this version

- **Testing:** The real test runs against actual websites still have to be done by whoever runs this locally.
- **Crawling manners:** There's no robots.txt or per-site rate awareness yet, the concurrency limit is just a rough stand-in for being polite to a website.
- **File types:** Content that only exists in a PDF or an image on a page isn't read in this version.
- **Big pages:** Each page or batch gets just one AI call to read it, a page with a lot of content is read within that one call's limit, not split into smaller pieces.